COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	July 16, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
    Student(std::string newName="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout<<"Constructor Called"<<std::endl;
    };

    //destructor
    ~Student(){
        std::cout<<"Destructor Called"<<std::endl;
    };

    //copy constructor
    Student(const Student &copyStudent){
        std::cout<<"Copy Constructor Called"<<std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    };

    //Display Attributes
    void printDetails(){
        std::cout<<this->studentName<<" "<<this->studentAge<<std::endl;
    };
};
```

```
33
34
35  int main() {
36
37      Student student1("Roman",28);
38      Student student2(student1);
39      Student student3;
40      student3 = student2;
41
42      return 0;
43
44  }
```

Output:

```
Constructor Called
Copy Constructor Called
Constructor Called
Destructor Called
Destructor Called
Destructor Called
PS C:\Users\TIPQC>
```

Observation	After creating the private and public class that contains the constructor, which contains the newName “John Doe as its default and newAge as 18 that then prints out “constructor called”, the destructor that deletes the previous object, the copy constructor, which copies another object, and the printDetails which pretty much prints out the variables. In the main function, the student 1 object is created by calling the constructor that then sets student 1’s name as “Roman” and its age as 28. The second object is student 2. Instead of calling the constructor, it calls the copy constructor. It sets the name of student 2 the same as student1. So, the name of student 2 is also “Roman” and their age is also 28. The third object is student 3 which calls the constructor. Since no name or age is assigned, student 3 uses the default name which is “John Doe” and the age 18.

	After that, student 3 is set as student 2. This does not call the copy constructor since it is manually assigned. After everything, the destructor is called which deletes all of the objects that were created. Since there was three objects created, the destructor is called three times.

Table 2-1. Initial Driver Program

Screenshot

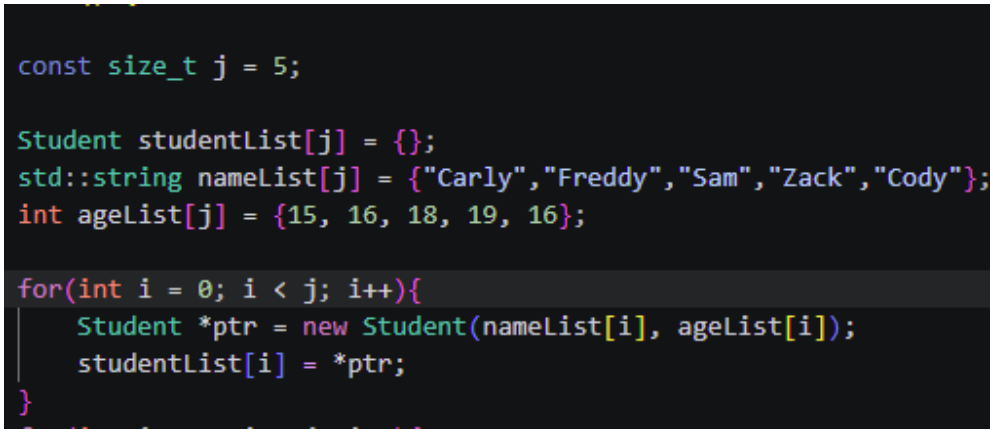
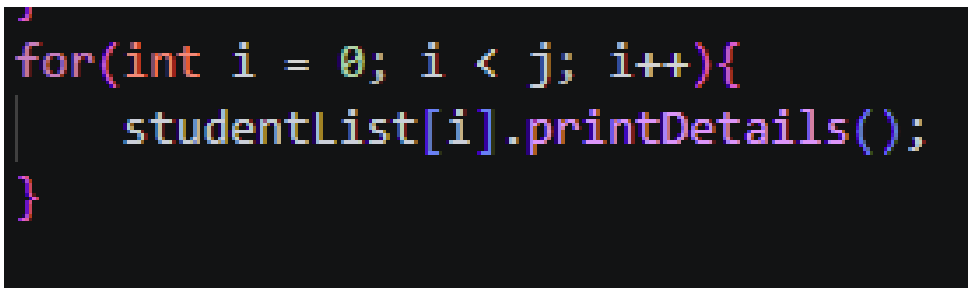
```
33
34
35  int main() {
36
37      const size_t j = 5;
38
39      Student studentList[j] = {};
40      std::string nameList[j] = {"Carly","Freddy","Sam","Zack","Cody"};
41      int ageList[j] = {15, 16, 18, 19, 16};
42
43      return 0;
44
45  }
```

Output:

```
Constructor Called
Constructor Called
Constructor Called
Constructor Called
Constructor Called
Destructor Called
Destructor Called
Destructor Called
Destructor Called
Destructor Called
```

Observation	First, we set 5 as a constant for the array size. The program then creates five objects in the studentList array. Since the array size is five, the constructor is called five times as well. If we display it, the array will contain five objects with each named “John Doe” and each has the age 18.The second array is named nameList with five names. Another array is created named ageList with five ages. The arrays nameList and ageList do not call a constructor since they are created separate from the studentList. After, the destructor is called. Since five objects are called in the studentList array, the destructor is also called five times.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	Same as the last item, the studentList is created by calling the constructor and creating five objects with the student name as “John Doe” and age as 18. Another list is created named nameList that lists five names and ageList which lists five ages. For the first loop, it creates a new student object on the heap. Each iteration calls the constructor and assigns the name based on nameList and ageList index. So, the first object will be named “Carly” with the age of 15, the second object will be named “Freddy” with the age 16, and so on. The next line is the studentList[i] = *ptr. This dereferences the pointer, so instead of referring to the pointer, it will refer to the object inside it. So, the constructor will be called ten times, five times when creating the studentList array and five times when creating the object for each iteration using the for loop.
Loop B	
Observation	This for loop displays the detail for each object. So, it will display “Carly” with age 15, the second object that will display is named “Freddy” with the age 16, and so on. In total, the

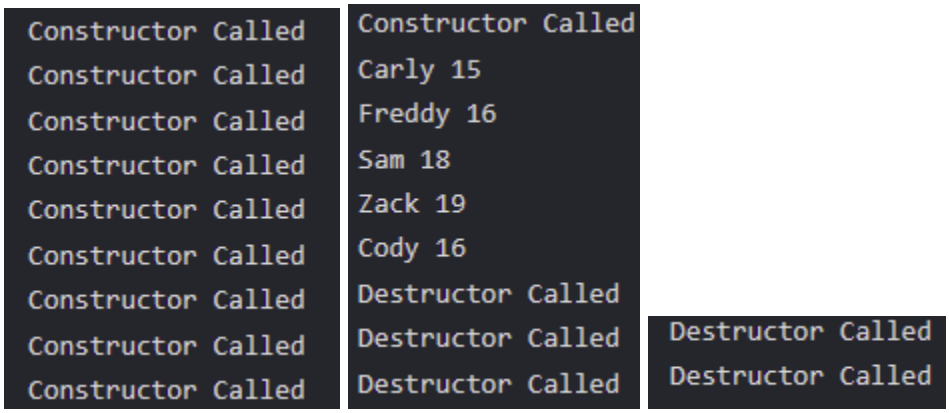
	program will display five names and ages. After everything is displayed, the destructor will be called five times and will delete all the objects that were created.
Output	
Observation	The constructor was called ten times because five objects were created for the studentList array. Another five objects were created when loop A is used. After the studentList is updated with the new objects, the content of each is displayed with each name and age. After, the destructor is called to delete each object. Since five objects are created, the destructor also deletes five objects.

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

PROBLEM 1

```cpp
#include <iostream>
#include <string.h>

class Fruits{
private:
    std::string productName;
    double price;
    int quantity;
public:
    //constructor
    Fruits(std::string newProductName = "", double newPrice = 0, int newQuantity = 0){
        productName = newProductName;
        price = newPrice;
        quantity = newQuantity;
    };

    //destructor
    ~Fruits(){
        std::cout<<"Destructor Called"<<std::endl;
    };

    //copy constructor
    Fruits(const Fruits &copyFruit){
        productName = copyFruit.productName;
        price = copyFruit.price;
        quantity = copyFruit.quantity;
    };

    //copy assignment
    Fruits& operator=(const Fruits &copyFruit){
        if(this != &copyFruit){
            productName = copyFruit.productName;
            price = copyFruit.price;
            quantity = copyFruit.quantity;
        }
        return *this;
    };

    //display
    void display(){
        std::cout<<"\nFruit: "<<productName<<std::endl;
        std::cout<<"Price: "<<price<<std::endl;
        std::cout<<"Quantity: "<<quantity<<std::endl;
        std::cout<<"Total: "<<calculateSum()<<std::endl;
    };

    double calculateSum(){
        return price * quantity;
    };

};
```

```cpp
class Vegetables{
private:
    std::string productName;
    double price;
    int quantity;
public:
    //constructor
    Vegetables(std::string newProductName = "", double newPrice = 0, int newQuantity = 0){
        productName = newProductName;
        price = newPrice;
        quantity = newQuantity;
    };

    //destructor
    ~Vegetables(){
        std::cout<<"Destructor Called"<<std::endl;
    };

    //copy constructor
    Vegetables(const Vegetables &copyVegetables){
        productName = copyVegetables.productName;
        price = copyVegetables.price;
        quantity = copyVegetables.quantity;
    };

    //copy assignment
    Vegetables& operator=(const Vegetables &copyVegetables){
        if(this != &copyVegetables){
            productName = copyVegetables.productName;
            price = copyVegetables.price;
            quantity = copyVegetables.quantity;
        }
        return *this;
    };

    //display
    void display(){
        std::cout<<"\nVegetables: "<<productName<<std::endl;
        std::cout<<"Price: "<<price<<std::endl;
        std::cout<<"Quantity: "<<quantity<<std::endl;
        std::cout<<"Total: "<<calculateSum()<<std::endl;
    }

    double calculateSum(){
        return price * quantity;
    };

};
```

```cpp
double totalSum(Fruits* fruits[], int fruitsCount, Vegetables* vegetables[], int vegetablesCount){

    double total = 0;

    for(int i = 0; i < fruitsCount; i++){
        total += fruits[i]->calculateSum();
    }

    for(int i = 0; i < vegetablesCount; i++){
        total += vegetables[i]->calculateSum();
    }

    return total;
};
```

PROBLEM 2

```cpp
int main(){
    Fruits* apple = new Fruits("Apple",10,7);
    Fruits* banana = new Fruits("Banana",10,8);
    Vegetables* broccoli = new Vegetables("Broccoli",60,12);
    Vegetables* lettuce = new Vegetables("Lettuce",50,10);

    //grocery list
    Fruits* fruitList[2];
    Vegetables* vegetableList[2];

    fruitList[0] = apple;
    fruitList[1] = banana;
    vegetableList[0] = broccoli;
    vegetableList[1] = lettuce;

    //grocery list display
    std::cout<<"===Grocery List==="<<std::endl;

    for(int i=0; i < 2; i++){
        fruitList[i]->display();
    }

    for(int i=0; i < 2; i++){
        vegetableList[i]->display();
    }

    std::cout<<"\n";
```

```
132
133     std::cout<< "Grand Total = PHP "<<totalSum(fruitList, 2, vegetableList, 2)<<std::endl;
134
135     std::cout<<"\n";
```

PROBLEM 3

```
102 double totalSum(Fruits* fruits[], int fruitsCount,
103                 Vegetables* vegetables[], int vegetableCount);
104
```

```
160 double totalSum(Fruits* fruits[], int fruitsCount, Vegetables* vegetables[], int vegetablesCount){
161
162     double total = 0;
163
164     for(int i = 0; i < fruitsCount; i++){
165         total += fruits[i]->calculateSum();
166     }
167
168     for(int i = 0; i < vegetablesCount; i++){
169         total += vegetables[i]->calculateSum();
170     }
171
172     return total;
173 };
```

PROBLEM 4

```
136
137     //delete lettuce
138     std::cout<<"Deleting Lettuce From The List"<<std::endl;
139     delete vegetableList[1];
140     vegetableList[1] = NULL;
141
142     //remaining
143     for(int i=0; i < 2; i++){
144         fruitList[i]->display();
145     }
146
147     if(vegetableList[0] != NULL){
148         vegetableList[0]->display();
149     }
150
151     std::cout<<"\n";
152
153     delete fruitList[0];
154     delete fruitList[1];
155     delete vegetableList[0];
156
157     return 0;
158 }
```

OUTPUT

```
===Grocery List===

Fruit: Apple
Price: 10
Quantity: 7
Total: 70

Fruit: Banana
Price: 10
Quantity: 8
Total: 80

Vegetables: Broccoli
Price: 60
Quantity: 12
Total: 720

Vegetables: Lettuce
Price: 50
Quantity: 10
Total: 500

Grand Total = PHP 1370

Deleting Lettuce From The List
Destructor Called
```

```
Fruit: Apple
Price: 10
Quantity: 7
Total: 70

Fruit: Banana
Price: 10
Quantity: 8
Total: 80

Vegetables: Broccoli
Price: 60
Quantity: 12
Total: 720

Destructor Called
Destructor Called
Destructor Called

--------------------------------
Process exited after 0.2606 seconds with return value 0
Press any key to continue . . .
```

PROBLEM 5

For the first problem, we create two classes, one for the fruits and one for the vegetables. The classes contains a private and a public class. The private class contains the variables needed for the program which are the product name, price, and quantity. The public class contains a constructor, a destructor, a copy constructor, and a copy assignment operator. The constructor is called when an object is created, it initializes the object's data members, setting the product name, price, and quantity based on the given variables. The destructor is called when deleting an object. The copy constructor creates a new object from an existing object while the copy assignment operator copies the content of an existing object to another existing object. I also created functions which are the display, which displays all the information, and calculateSum, which calculates the sum for each class. For the second problem, I set up the grocery list array. First, we create the object for each product using dynamic allocation. Then, I created an array for the fruits and vegetables. I set each index of the array as the object I created using dynamic allocation. After, I display the content of each list using a for loop that goes through the fruit and vegetable list using "->" to access variables via pointers and calling the display to display all the information of the product. We display the grand total by using the totalSum function. On the third problem, this is where I create the totalSum function, which calculates the total sum for the fruits and for the vegetables. It does this by declaring a variable, total, and setting it to 0. Then, using a for loop to go to the fruits list array and adding it to the total and then, going through the vegetable list array and also adding it to the total. It returns the total. For the fourth problem, I delete the lettuce from the vegetable list by using the delete function which calls our destructor. After, we point it to the null. I display the remaining list by using the same for loop. For displaying the vegetable list, I use an if statement. If we use a for loop, it will try to display the null pointer which is not a real object. So, instead, I use the if statement to check if the remaining object is a real object or not. If it is, it calls the display function.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: N/A

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: N/A

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: N/A

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.